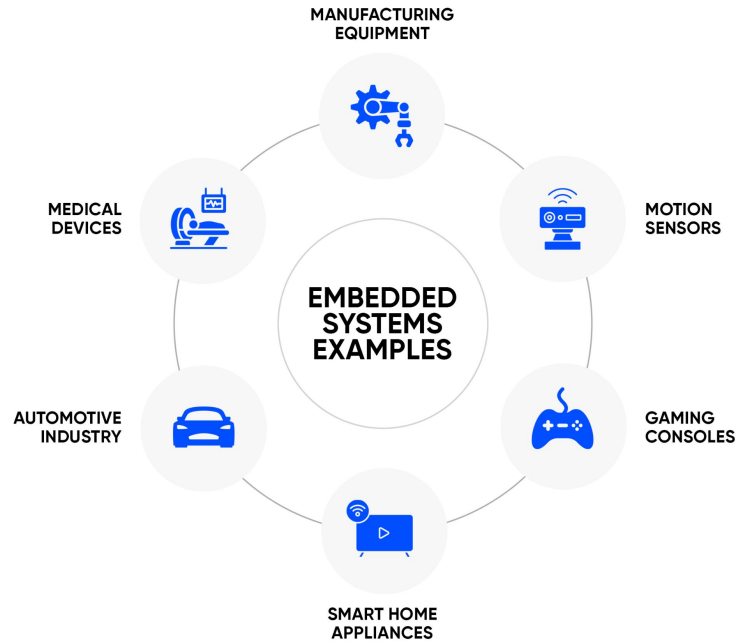


**CSE - 3106**

# Embedded System

Dedicated computer systems, often operating in real-time, that control a specific function within a larger device.

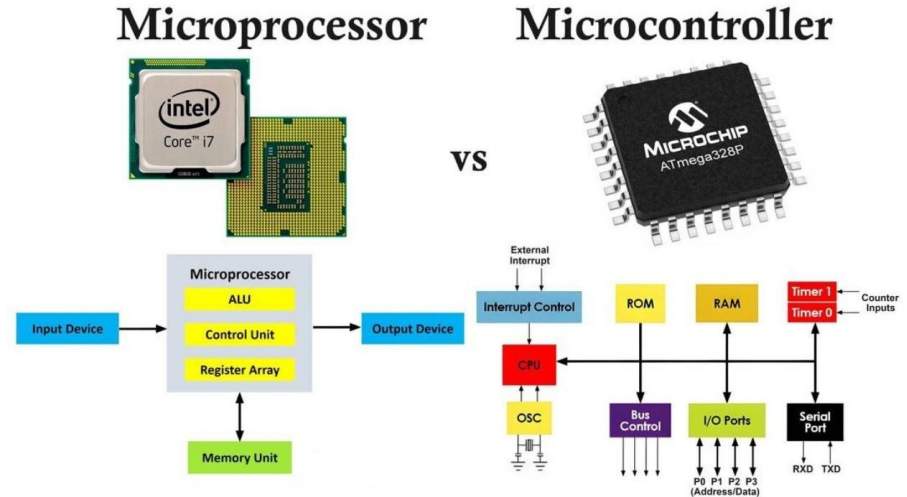
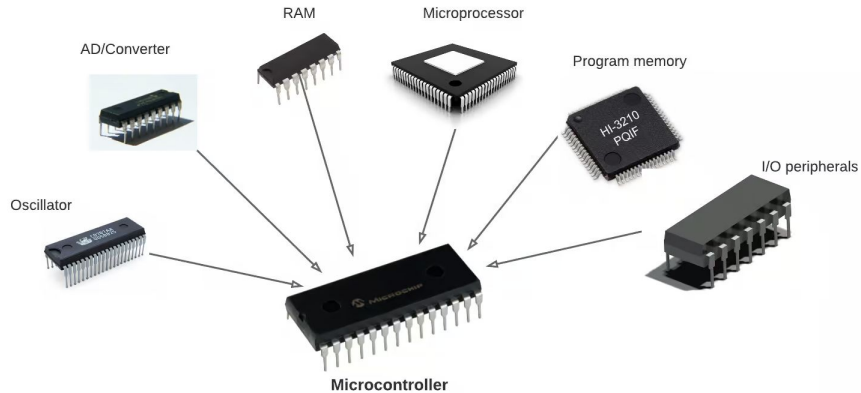


## Applications for Embedded Systems

# Microcontrollers vs. Microprocessor

**Microcontroller:** A compact integrated circuit designed to **govern a specific operation in an embedded system**. It contains a processor, memory, and I/O peripherals on a single chip. Optimized for Control

**Microprocessor:** A microprocessor is a single integrated circuit (chip) that contains the **entire Central Processing Unit (CPU)** of a computer, performing all arithmetic, logic, and control functions.



# Popular Boards

Arduino, ESP32, Raspberry Pi



Arduino

# Popular Boards

Arduino, ESP32, Raspberry Pi



ESP32

# Popular Boards

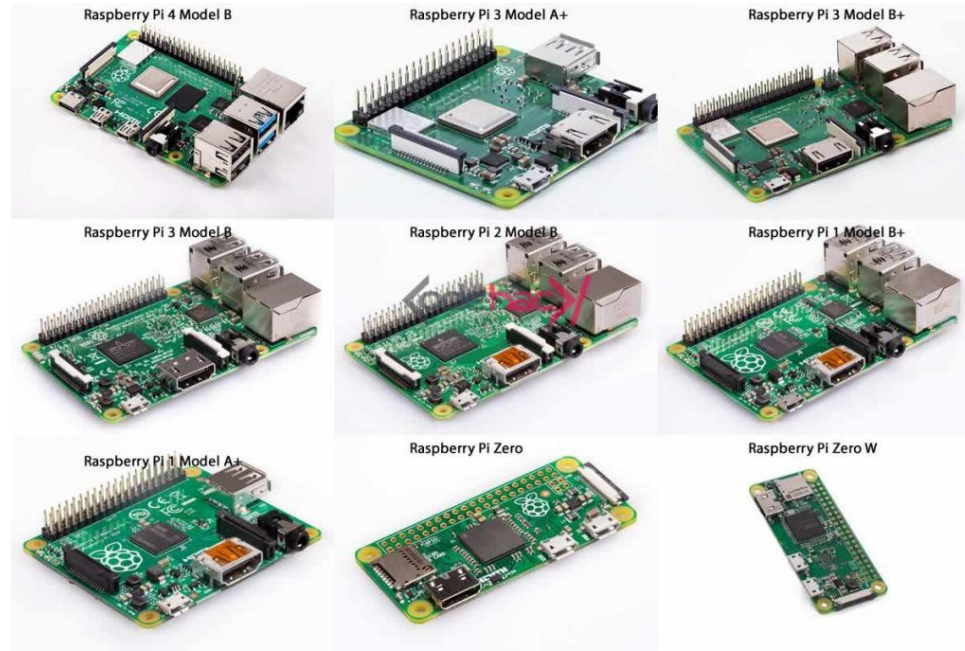
Arduino, ESP32, Raspberry Pi



**ESP32**

# Popular Boards

Arduino, ESP32, Raspberry Pi



**Raspberry Pi**

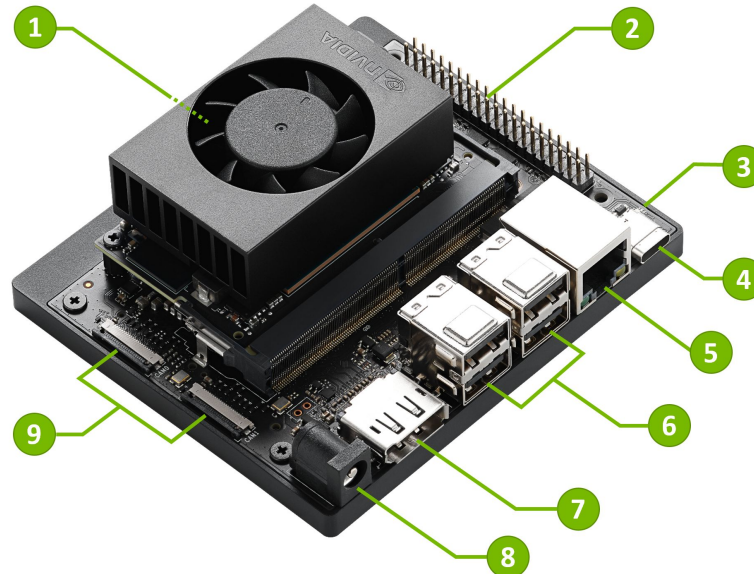
# Which One to choose?

**Arduino:** Simple Tasks => LED Blinking, Simple Motor Control || **Runs C/Python**

**ESP32:** IoT Projects, Requires Wi-Fi/Bluetooth => Weather Station, Remote Control Switch || **Runs C/C++, Python/ Micropython**

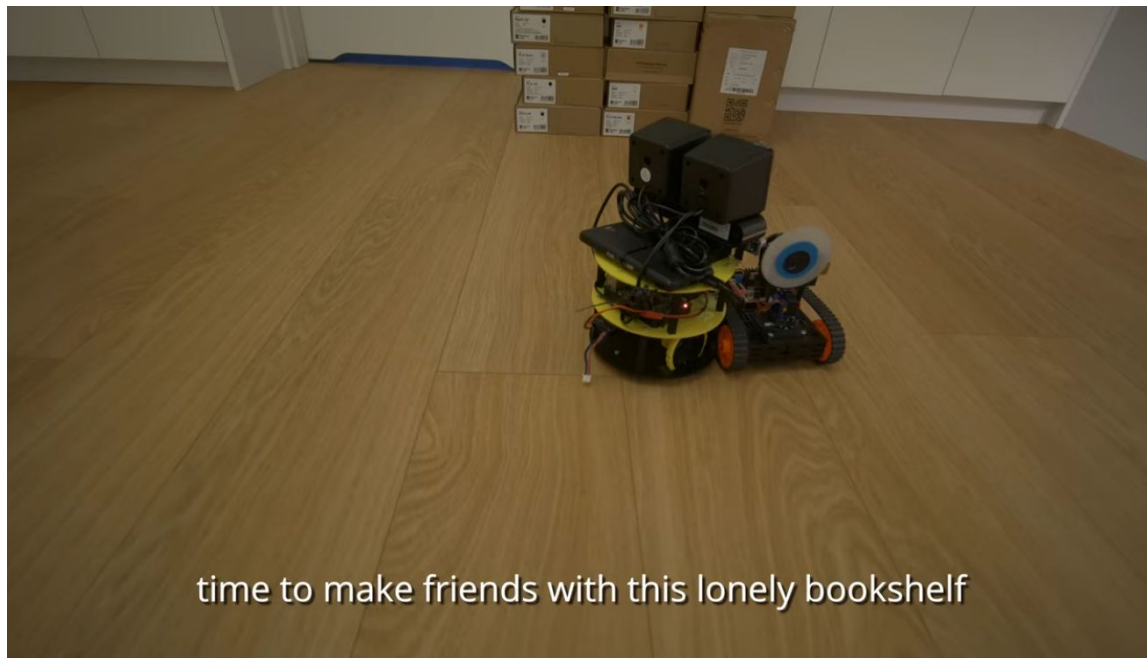
**Raspberry Pi:** Complex Tasks that requires Heavy Processing => Home Automation, Small Robots, Media Hub || **Runs C/C++, Python/ Micropython, OS**

**Jetson Nano:** Single-Board Computer (Microcomputer) Designed for AI/ML tasks



Nvidia Jetson Nano

## Which One to choose?



time to make friends with this lonely bookshelf

[This mini GPU runs LLM that controls this robot](#)

## Running Code



Arduino IDE



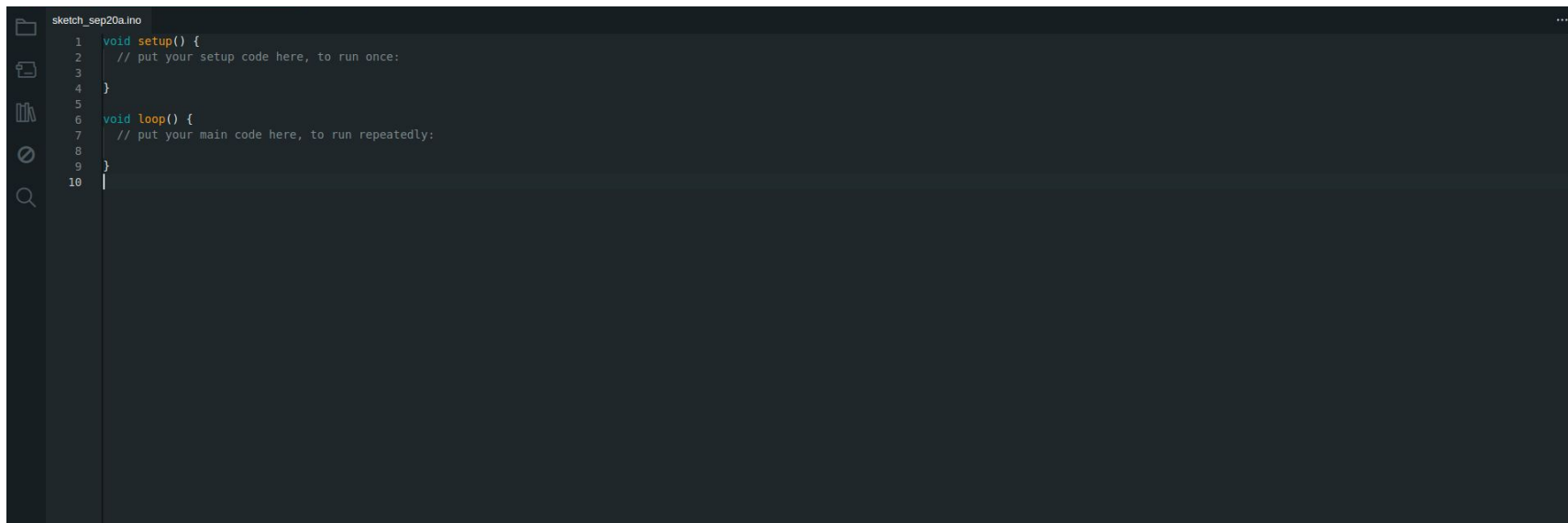
Simulation

# Running Code

An Arduino program is referred to as a **Sketch** and is saved with the **.ino** file extension.

The **.ino** file is the core file containing main logic, including the required **setup()** and **loop()** functions.

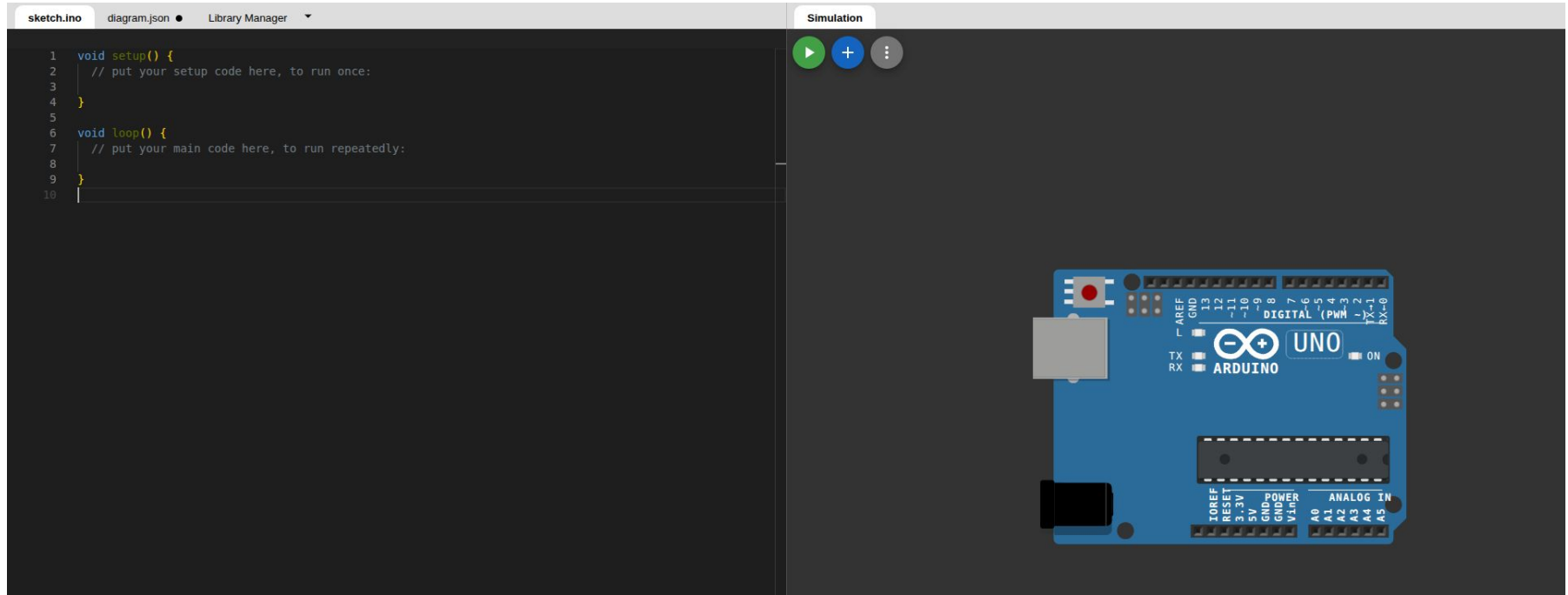
The IDE handles the complexity of adding necessary files and libraries for you, simplifying the coding process.



```
sketch_sep20a.ino
1 void setup() {
2   // put your setup code here, to run once:
3
4 }
5
6 void loop() {
7   // put your main code here, to run repeatedly:
8
9 }
10
```

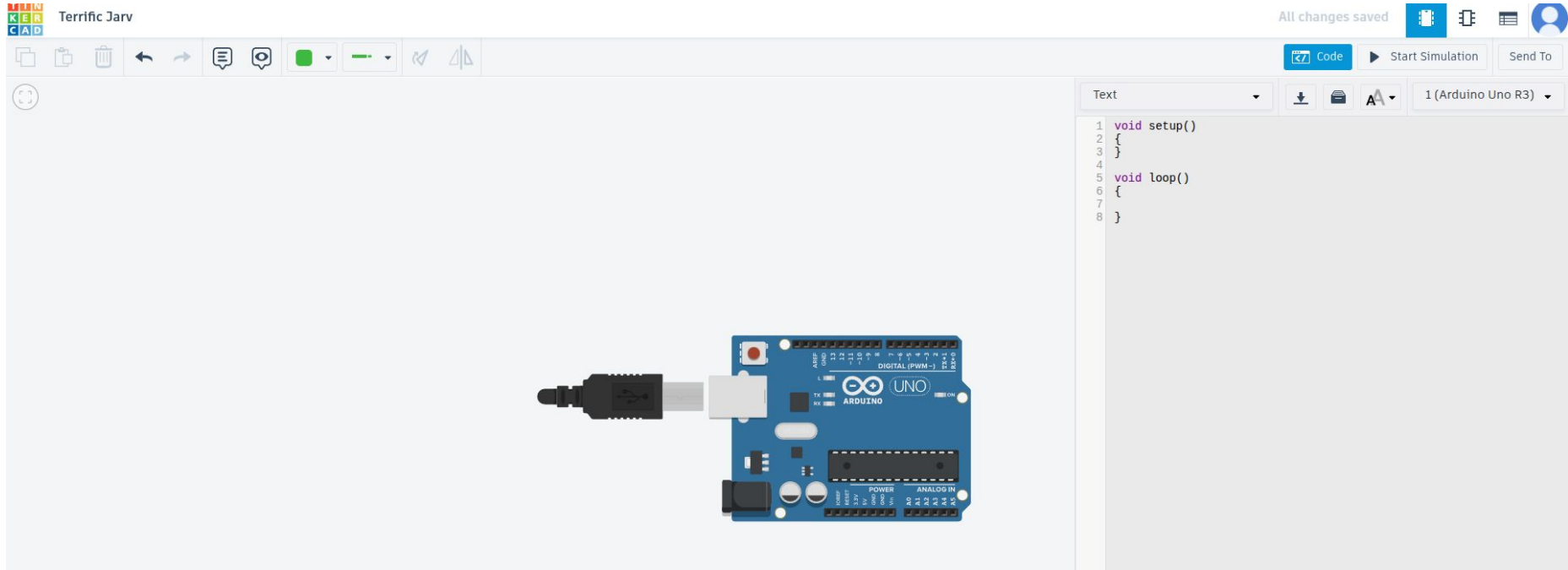
Arduino IDE

# Running Code



WOKWI Interface

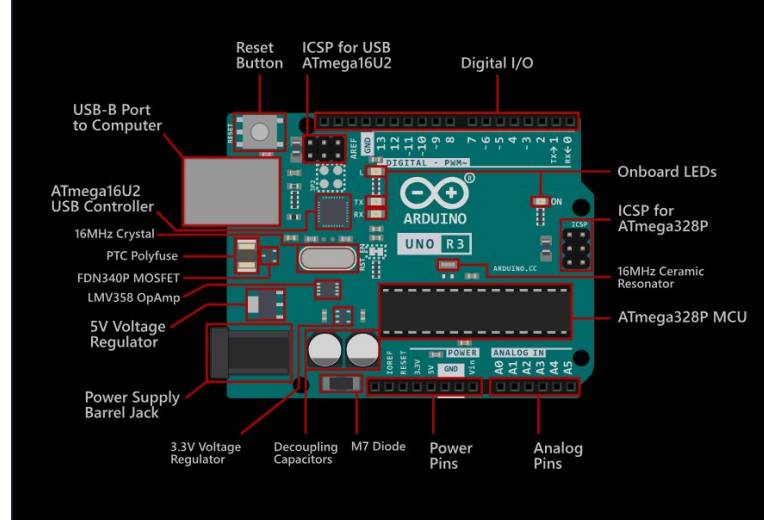
# Running Code



Tinkercad Interface

# Arduino UNO R3

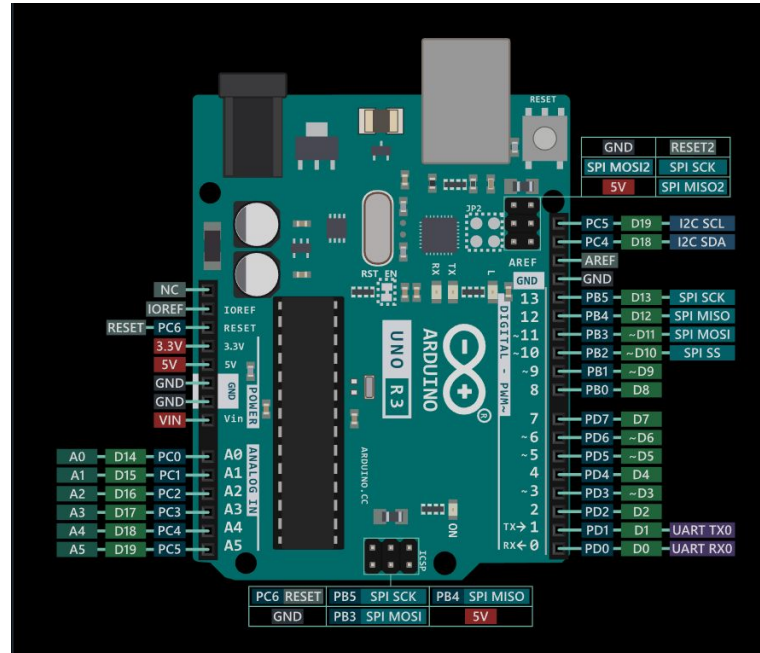
- The Uno R3 uses the **ATmega328P** microcontroller,
- Runs at 16 MHz and operates at a 5V logic level.
- Ideal for learning and simple control tasks.



Arduino Uno R3 Board Components

# Arduino UNO R3

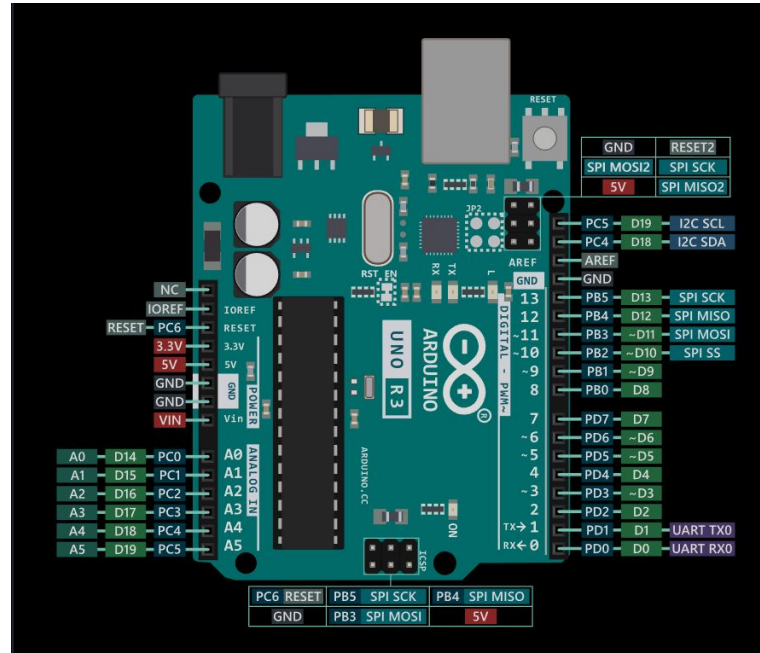
- **Digital I/O Pins (14):** Pins 0 to 13 can be used as either inputs or outputs.
- **PWM Pins (6):** Pins 3,5,6,9,10,11 are marked with a (~) and can produce **Pulse Width Modulation** (like analog output for LEDs/motors).
- **Analog Input Pins (6):** Pins A0 to A5 are for reading continuous voltage signals from analog sensors.



Arduino Uno R3 Pinout

# Arduino UNO R3

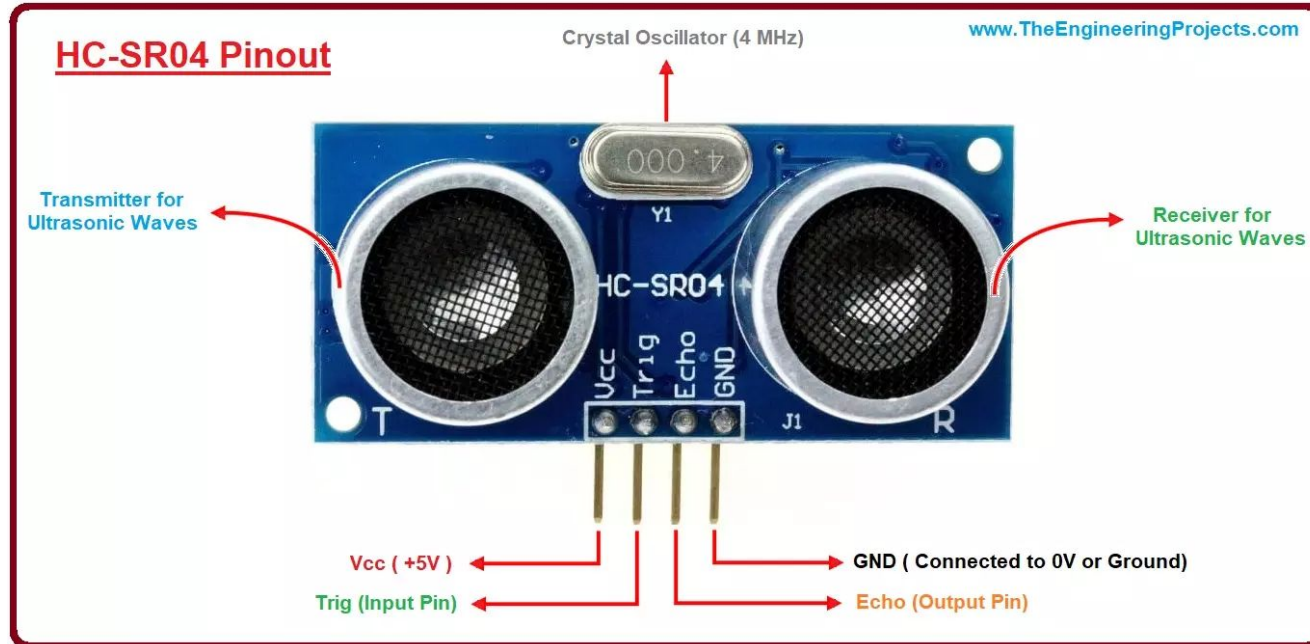
- **Power Pins (5V, 3.3V):** Regulated outputs to power sensors and components.
- **GND Pins:** Multiple **Ground** pins for completing the circuit.
- **VIN Pin:** Allows the board to be powered from an external 7V to 12V power supply.



Arduino Uno R3 Pinout

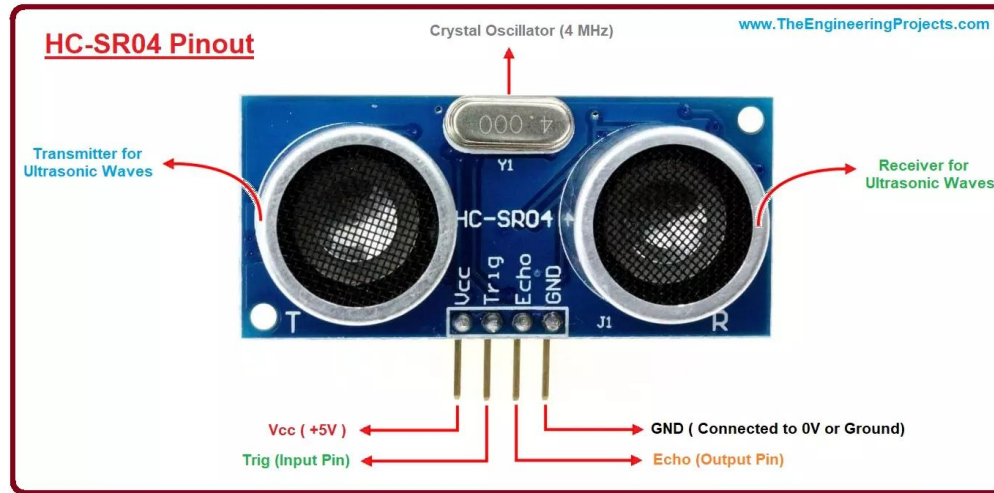
# HC-SR04 Ultrasonic Sensor

- Measures distance by emitting a high-frequency 40 kHz sound pulse and listening for the echo.
- It uses the formula **Distance = (Speed of Sound × Time) / 2** to calculate the distance to an object.
- It's commonly used in robotics for obstacle avoidance and in measurement systems.



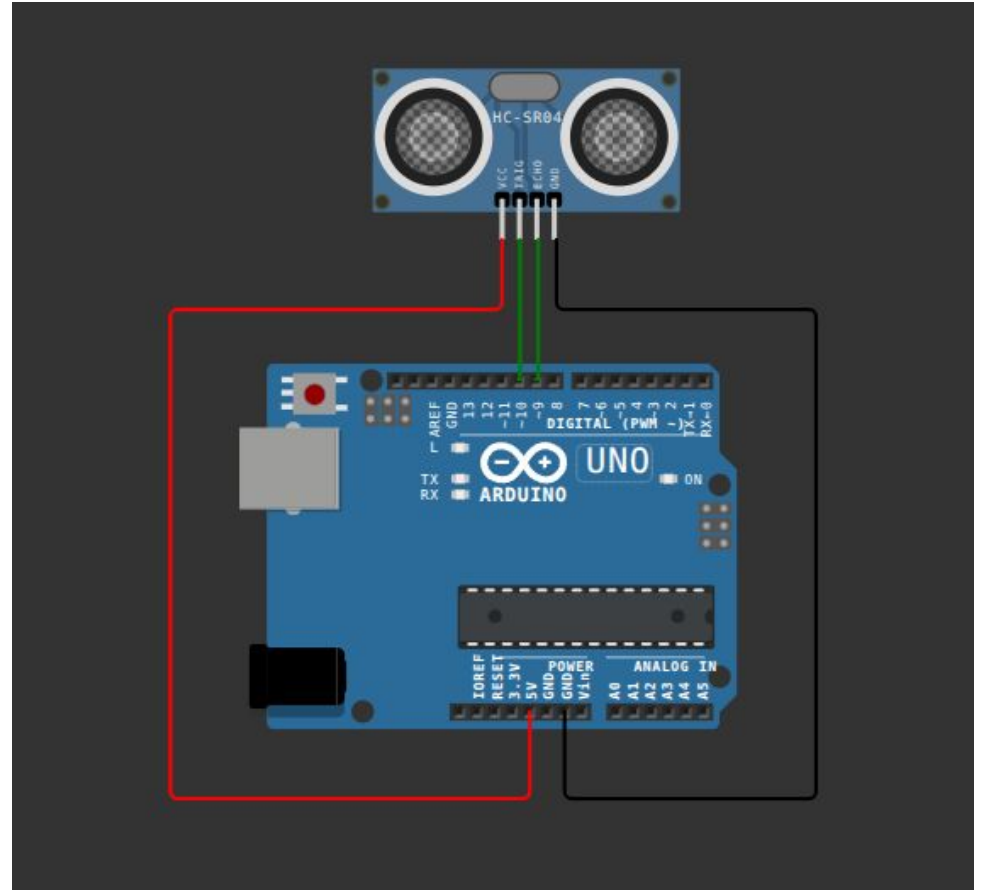
# HC-SR04 Ultrasonic Sensor

- **VCC:** Power supply pin, connected to the Arduino's 5V.
- **GND:** Ground pin, connected to the Arduino's GND.
- **Trig (Trigger):** An Input pin; a short pulse here tells the sensor to send a sound wave.
- **Echo:** An Output pin; the duration of its HIGH pulse is equal to the time the sound wave took to return.



# Wiring Diagram

- VCC on sensor → **5V** on Arduino.
- GND on sensor → **GND** on Arduino.
- Trig on sensor → Digital Pin **10** on Arduino.
- Echo on sensor → Digital Pin **9** on Arduino.



# Simulation

```
// Pin Definitions  
const int TRIG_PIN = 10;  
const int ECHO_PIN = 9;
```

```
void setup() {  
  pinMode(TRIG_PIN, OUTPUT);  
  pinMode(ECHO_PIN, INPUT);  
  Serial.begin(9600);  
}
```

```
void loop() {
```

```
  long duration_us;  
  float distance_cm;
```

```
  digitalWrite(TRIG_PIN, HIGH);  
  delayMicroseconds(10);  
  digitalWrite(TRIG_PIN, LOW);
```

```
  duration_us = pulseIn(ECHO_PIN, HIGH);
```

```
  distance_cm = duration_us * 0.034 / 2;
```

```
  Serial.print("Distance: ");  
  Serial.print(distance_cm);  
  Serial.println(" cm");
```

```
  delay(500);  
}
```

```
// Trigger pin to output (send)  
// Echo pin to input (listen)
```

```
// Time in microsecond  
// Distance in cm
```

```
// Send Ultrasonic Pulse
```

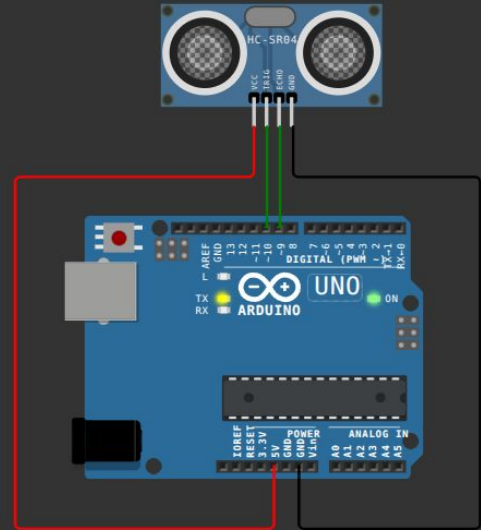
```
// Measure echo-pulse duration
```

```
// Calculate distance. Speed of sound = 0.034 cm/us
```

```
// Print Results
```

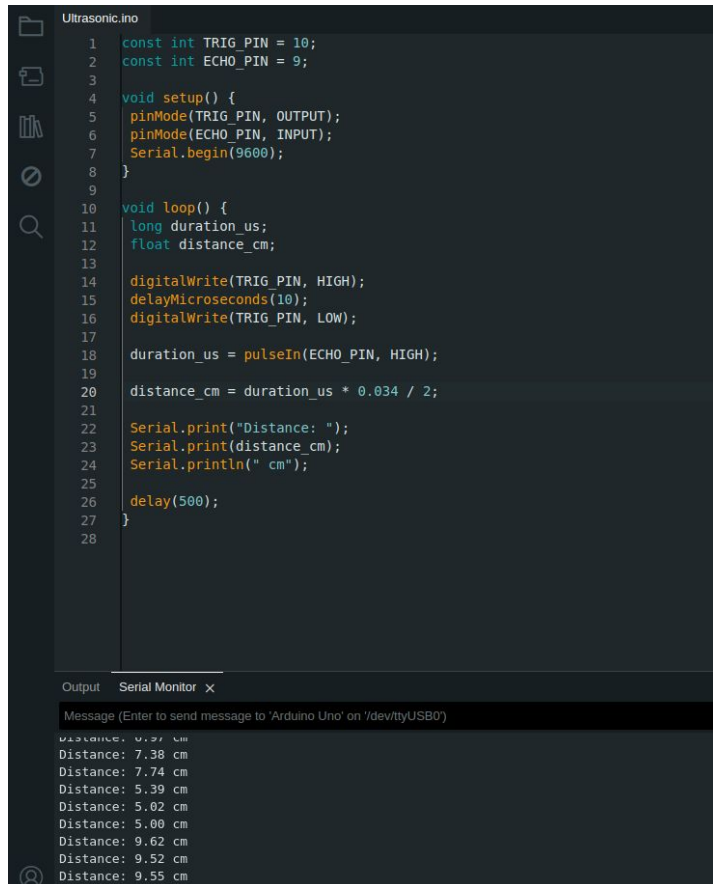
Simulation

00:02.738 100%



```
Distance: 397.65 cm  
Distance: 397.65 cm  
Distance: 397.63 cm  
Distance: 397.63 cm  
Distance: 397.65 cm  
Distance: 397.65 cm
```

# Arduino IDE



The screenshot shows the Arduino IDE interface. The main editor window displays a C++ program named 'Ultrasonic.ino'. The code defines two pins, TRIG\_PIN (10) and ECHO\_PIN (9), and implements a setup and loop function. The loop function sends a pulse to the TRIG\_PIN, measures the duration of the echo, and calculates the distance in centimeters. The distance is printed to the serial monitor every 500 milliseconds. Below the code editor, the 'Serial Monitor' window is open, showing a series of distance measurements in centimeters.

```
Ultrasonic.ino
1  const int TRIG_PIN = 10;
2  const int ECHO_PIN = 9;
3
4  void setup() {
5    pinMode(TRIG_PIN, OUTPUT);
6    pinMode(ECHO_PIN, INPUT);
7    Serial.begin(9600);
8  }
9
10 void loop() {
11   long duration_us;
12   float distance_cm;
13
14   digitalWrite(TRIG_PIN, HIGH);
15   delayMicroseconds(10);
16   digitalWrite(TRIG_PIN, LOW);
17
18   duration_us = pulseIn(ECHO_PIN, HIGH);
19
20   distance_cm = duration_us * 0.034 / 2;
21
22   Serial.print("Distance: ");
23   Serial.print(distance_cm);
24   Serial.println(" cm");
25
26   delay(500);
27 }
28
```

Output Serial Monitor X

Message (Enter to send message to 'Arduino Uno' on '/dev/ttyUSB0')

Distance: 0.00 cm  
Distance: 7.38 cm  
Distance: 7.74 cm  
Distance: 5.39 cm  
Distance: 5.02 cm  
Distance: 5.00 cm  
Distance: 9.62 cm  
Distance: 9.52 cm  
Distance: 9.55 cm

## Task + Display

1. Automatic Hand Sanitizer: Use an IR sensor to detect a hand and trigger a servo motor to dispense sanitizer.
  2. Digital Thermostat: Read ambient temperature from a DHT11 sensor and display it on an LCD, switching an LED (simulating a fan) if the temperature is too high.
  3. Smart Plant Watering: Read soil moisture; if it's too dry, briefly activate a small pump to water the plant.
  4. Light-Sensitive Night Light: Use an LDR to measure darkness and automatically turn on a bright LED when the light level drops.
  5. Garage Door Status Indicator: Use a limit switch to check if a door is closed; if open, sound a buzzer and light a red LED.
  6. Security Tripwire Alarm: When a PIR motion sensor detects movement, activate a buzzer for 5 seconds to alert the user.
- 
7. Interactive Sound Board: Use three separate pushbuttons to trigger and play three different tones/pitches on a Piezo buzzer.
  8. Digital Spirit Level: Read the tilt angle from an MPU-6050 and display the pitch and roll degrees on an OLED screen.
  9. Password Door Lock: Enter a code using a keypad; if correct, momentarily move a servo motor to 'unlock' a latch.
  10. Simple Bar Graph Meter: Use a potentiometer to adjust a value, which is then visually represented by lighting up a corresponding number of LEDs in a bar graph.